Phase 0: Foundation (before touching code)

1. Freeze a single research question in writing.
Why: Right now the project has tried to answer three different questions (uniform vs DRL, Byzantine defense, security). Pick one main question — "when does adaptive privacy budgeting help?" — and treat everything else as secondary. This stops scope creep and gives you one thing to defend.

2. List every number currently in the thesis and mark which ones have a data file behind them.
Why: The review found numbers with no traceable source, and one plotting script that fabricated curves. Any untraceable number is a risk at defence. Delete or re-earn each one.

3. Decide your dataset split rule before writing any code: split by recording session, never by frame position.
Why: Two different leaks already happened this way (train/test and validation/test). Deciding the rule first, in writing, stops it happening a third time.

Phase 1: Build the measurable pipeline

4. Pick the lightweight architecture (frozen backbone + trainable head, e.g. MobileNet/StarNet backbone with a small head) and get it training end-to-end on one client, no FL yet.
Why: You need one working baseline before adding federated complexity. Debugging FL and DP and RL all at once is how the first round failed.

5. Run a noise sweep (σ = 0, 2, 4, 8, 16, 32) on this single-client model with no FL.
Why: Your group already discovered that freezing too much of the model can make noise have *zero* effect, even at huge σ. You need to confirm noise actually changes accuracy before building anything on top of it. If it's flat, unfreeze more layers or lower batch size until it isn't.

6. Report ε (not just ρ) for every noise level you plan to use, using a proper accountant (e.g. Opacus RDP).
Why: Reviewers care about ε, and your earlier draft never reported it. Rounds spent at $\varepsilon \approx 67$ are not meaningfully private; this needs to be visible from the start, not patched in later.

Phase 2: Turn on federated learning

7. Stand up the 5-client (or 10-client) FL setup, each client on a different environment split (clear sky, trees, buildings, etc.), same shared architecture.

Why: This is your non-IID realism. One shared architecture matters because federated averaging only works if all clients hold the same model shape — "different CNN per client" (as first suggested) breaks averaging.

8. Add FedProx.
Why: FedProx keeps each client's local update from drifting too far from the global model, which helps when clients hold very different data (different backgrounds). This is a cheap, well-understood addition.

9. Run and record four variants: FL+DP, FL without DP, DP without FL (single client), and neither.
Why:test This is the ablation your friend suggested. It shows, separately, what FL costs you and what DP costs you, instead of one tangled number.

Phase 3: Find where the noise actually matters

10. Run the noise-pulse test (E1): add one burst of extra noise in a single round, measure the accuracy drop, repeat for several rounds and a couple of σ levels, 10+ paired seeds.
Why: This single experiment tells you whether late-round noise hurts more than early-round noise in *your* setup. It is the experiment that settles the "does schedule matter here" argument with data instead of opinion.

11. From that plot, choose an operating σ where the schedule can plausibly matter (not near the floor, not near the ceiling).
Why: Your pre-defence numbers sat at mAP 0.93–0.99 — too close to perfect for any method to show a real difference. Calibrating into a harder regime is a precondition for any comparison meaning anything.

Phase 4: The main comparison (your actual thesis result)

12. Build four allocation arms spending the identical total budget: Uniform, tuned exponential decay, "blind" DRL (round+budget only, like Chen et al.), and feedback DRL (adds loss/remaining-budget, with a GRU/LSTM so it can track trends over rounds).
Why: This is the fair test. Arm 3 reproduces the paper's weak setup on purpose, so you can show *why* it fails. Arm 4 is your actual improvement attempt.

13. Switch from fixed clipping to adaptive (per-round) clipping for the DRL arms.
Why: The "uniform is optimal" proof only holds when clipping is fixed. Adaptive clipping removes that assumption, which is exactly the condition under which a learned schedule has something real to exploit.

14. Write down your win/tie/loss decision rule before running the comparison.
Why: Deciding in advance what counts as a win stops you (or a skeptical teammate) from arguing about the result after seeing it. Science reviewers like this; it's called pre-registration.

15. Run 10+ paired trials per arm, same random seed shared across arms (common random numbers), on session-disjoint held-out data.
Why: Paired trials with shared randomness cut the noise in your measurements dramatically — your own project already proved this (2.7× tighter) — so you need far fewer runs to get a trustworthy answer.

16. Report mean difference, 95% confidence interval, and win count for every comparison, not just a single accuracy number.
Why: A single number can't be trusted at this scale of effect. Four single-run "wins" in this project already failed to replicate — the CI and win-count protect you from reporting noise as a result.

Phase 5: Security (Byzantine) side — do only after Phase 4 is solid

17. Fix the validation/test leak: carve out a session-disjoint root set for defenses, separate from both train and the final test set.
Why: This is finding M-4 — your earlier security numbers are unsafe because the "clean" validation data overlapped with the test data.

18. Swap Krum for FLTrust as your main aggregator.
Why: Classical robust aggregators (Krum, median) throw away the averaging that cancels DP noise, and in your own data they cost *more* accuracy than the attacks they defend against. FLTrust keeps more of the averaging.

19. Test against ALIE, IPM, and Fang attacks, not just random noise attacks.
Why: Random attacks get caught by almost anything, so they don't prove a defense works. These three are designed to evade detection, so beating them is a real claim.

communication rounds baray e dekha

Suggestions and things to keep in mind to make the research powerful

1. Compare against the real Q1 paper, not just Chen et al.

The earlier review flagged a paper (Kiani et al., ICLR 2025) that handles the same problem but with actual theory. If your result lines up with or improves on that paper instead of just beating a weak ICC paper, it's a much stronger comparison to put in front of a panel.
Why: Beating a flawed baseline isn't convincing. Beating (or matching) a respected one is.

2. Report the DRL agent's cost, not just its accuracy

Training a DRL agent takes extra compute and extra communication rounds. A tuned decay schedule costs almost nothing to deploy.
Why: Even if DRL ties or slightly beats the decay schedule, if it costs 10× more to train, that's a real finding worth stating plainly: "not worth it in practice." This turns a possible "tie" into a useful conclusion instead of a shrug.

3. Plot the schedule the agent actually learned against the theory's prediction

Your thesis has a theorem predicting the optimal shape ($\rho_t \propto \sqrt{a_t}$). After training the feedback DRL agent, plot its learned schedule next to that predicted curve.
Why: If they match, that's a strong, visual "theory and experiment agree" moment — the kind of figure that makes a defense panel sit up. If they don't match, that's also informative (the agent learned something theory didn't predict).

4. Count the agent's own training as a privacy cost

Training the RL agent runs many full DP-FL simulations on the same client data. That's privacy spending that's usually never counted.
Why: This is a subtle point the original paper missed entirely. Addressing it head-on (train on public/proxy data, or explicitly count it in your ε) is the kind of rigor that makes a thesis stand out, not just repeat others' mistakes.

5. Stress-test near the "cliff" (tight privacy budgets)

Your group already did a version of this (Section 5.1.5 in the paper you referenced) — run the comparison at a very small privacy budget, where noise is so strong that results get unstable.
Why: This is where your theorem's assumptions are weakest and where a real effect, if one exists, is most likely to show up. It also shows reviewers you tested the edges, not just the easy middle.

6. Keep a visible record of what didn't work

List the experiments and single-run results that looked promising but didn't survive repeating them, in an appendix.
Why: This sounds like a weakness but is actually a strength — it shows discipline and makes everything else you report more believable. Panels trust groups that show their failed attempts.

7. Write the decision rules and hypotheses down before running the main comparison

A one-page document: "we expect X, here's how we'll decide if we're right," written before Phase 4 starts.
Why: This is called pre-registration. It's a small amount of extra writing that makes every number afterward much harder to argue with — you can show you didn't pick the story after seeing results.

8. Test on a second dataset, even a small one (e.g., your CIFAR-10 port)

Even a rough second confirmation that the main finding holds outside the drone dataset.
Why: One dataset makes your result look like it might be an accident of that specific data. A second, even small, confirmation makes it look like a real pattern.

Analysis Part

What he's done vs. our plan

✅ Done — matches our workflow

Our step	What he did	Why it matters (easy words)
Step 3 (split rule) / Step 17 (fix leak)	Split data by recording identity, not folder. Found the old split leaked 70% of validation into training.	This was our #1 "fix before anything else" — a leaked test set makes every number after it untrustworthy. He actually fixed it.
Step 6 (report ε)	Reports ε ≈ 67 at δ=1e-5, not just ρ.	ρ alone doesn't mean anything to a privacy reviewer; ε does. He's showing the real privacy cost honestly.
Step 15 (paired trials, shared randomness)	Uses common-random-number paired trials (n=10–16), cutting noise from SD 0.0466 to 0.0171.	This is what lets you trust a small number of runs instead of needing hundreds.
Step 14 (decide rules before running)	States predictions were "pre-registered" before running, and reports two that failed.	Shows he's not just picking the results that look good after the fact — this is what makes a result believable to a panel.
Step 16 (report CI, not single numbers)	Gives confidence intervals, win counts, and flags "tight-budget gains are noise, n≈73 needed."	Protects against reporting a lucky single run as a real finding — exactly the trap your project fell into before.
Phase 6 (name the hidden assumption)	Explicitly states uniform is optimal "only if weights a□ are flat," tying it to the weighted theorem.	This is the corrected, honest version of the claim instead of the overstated "uniform always wins."
Core finding of Phase 4	Reproduced Chen et al.'s DRL win on MNIST (+2.27pp) and confirmed uniform wins at low noise, decay wins at high noise.	This is literally the central story we wanted the whole restart to prove. He already has it, independently.

"Don't repeat these mistakes" list	Separately documents: folder-leak, scoring-on-training-data inflation, meaningless single runs, Adam hiding noise, head-only DP being "free" noise, from-scratch training collapsing.	Same list of pitfalls we already flagged — he caught them himself through testing, which is a good sign.

❌ Not done — the gap in our plan

Our step	What's missing	Why it matters (easy words)
Step 12 (feedback DRL + GRU/LSTM)	His own notes say: "No RL agent using model feedback (loss or gradient norm as state)." Only the old "blind" Chen-style agent exists.	This is the actual new idea — a DRL that can see what's happening in training. Without it, we can't claim DRL ever truly outperforms a simple formula, because we never gave it a fair chance.
Step 13 (adaptive clipping)	Not mentioned anywhere in his work. Still uses fixed clipping.	The "uniform is optimal" proof only holds when clipping is fixed. Without adaptive clipping, DRL is mathematically boxed into losing or tying, no matter how smart it is.
FLTrust (friend's suggestion)	Still using Krum, median, trimmed mean — the same aggregators flagged as weak under DP.	FLTrust was suggested because it preserves more of the noise-cancelling averaging. Classical aggregators keep hurting accuracy in his own results (median costs −0.106 with no attack at all).
Second dataset check (bonus suggestion #8)	Only tested on one dataset/detector/architecture.	A result on one dataset could be a fluke of that data. No second check yet.

➕ Extra — he did things beyond our plan

What he did	Why it's extra (not in our steps)	Is it useful?
Full Byzantine security study with 5 real attacks (RMD, α-MPELM, band-exploiting, fixed-flip, σ-lying) and 6 defenses	Our plan put this in "Phase 5, only after Phase 4 is solid" — he ran it in parallel instead of after.	Yes, genuinely useful data — but it means effort went here instead of into the feedback-DRL experiment, which is the piece our main thesis claim actually needs.
Heterogeneous per-client noise sweep (cv(σ) at 3%, 6%, 13%, 25%)	Not something we specified — he invented this to test how uneven noise affects detection.	Interesting and relevant to "Claim 2" (noise allocation affects security), but it's a side-branch, not the core question.
Found that cosine-direction defenses stop working by round 13 ("honest cosine decays to 0.0003")	New finding, not asked for.	Important safety finding — shows a real weakness in a defense, worth keeping.
Checkpoint-sensitivity finding (DRL flips sign between checkpoint 114 vs 612, converges to a flat/useless schedule)	Goes beyond "does DRL win" into "when does it stop learning anything."	Very useful — it directly explains *why* blind DRL fails, which strengthens the case for trying feedback DRL next.